
1.9 Software Development

Traditional software development involves several phases. Three major steps are:

1. Design
2. Coding
3. Testing and Debugging

In this section, we briefly discuss the role of these phases, and we introduce several good practices for programming in Java, including coding style, naming conventions, formal documentation, and testing.

1.9.1 Design

For object-oriented programming, the design step is perhaps the most important phase in the process of developing software. It is in the design step that we decide how to divide the workings of our program into classes, when we decide how these classes will interact, what data each will store, and what actions each will perform. Indeed, one of the main challenges that beginning programmers face is deciding what classes to define to do the work of their program. While general prescriptions are hard to come by, there are some rules of thumb that we can apply when determining how to define our classes:

- **Responsibilities:** Divide the work into different *actors*, each with a different responsibility. Try to describe responsibilities using action verbs. These actors will form the classes for the program.
- **Independence:** Define the work for each class to be as independent from other classes as possible. Subdivide responsibilities between classes so that each class has autonomy over some aspect of the program. Give data (as instance variables) to the class that has jurisdiction over the actions that require access to this data.
- **Behaviors:** Define the behaviors for each class carefully and precisely, so that the consequences of each action performed by a class will be well understood by other classes that interact with it. These behaviors will define the methods that this class performs, and the set of behaviors for a class form the *protocol* by which other pieces of code will interact with objects from the class.

Defining the classes, together with their instance variables and methods, are key to the design of an object-oriented program. A good programmer will naturally develop greater skill in performing these tasks over time, as experience teaches him or her to notice patterns in the requirements of a program that match patterns that he or she has seen before.

A common tool for developing an initial high-level design for a project is the use of **CRC cards**. Class-Responsibility-Collaborator (CRC) cards are simple index cards that subdivide the work required of a program. The main idea behind this tool is to have each card represent a component, which will ultimately become a class in the program. We write the name of each component on the top of an index card. On the left-hand side of the card, we begin writing the responsibilities for this component. On the right-hand side, we list the collaborators for this component, that is, the other components that this component will have to interact with to perform its duties.

The design process iterates through an action/actor cycle, where we first identify an action (that is, a responsibility), and we then determine an actor (that is, a component) that is best suited to perform that action. The design is complete when we have assigned all actions to actors. In using index cards for this process (rather than larger pieces of paper), we are relying on the fact that each component should have a small set of responsibilities and collaborators. Enforcing this rule helps keep the individual classes manageable.

As the design takes form, a standard approach to explain and document the design is the use of UML (Unified Modeling Language) diagrams to express the organization of a program. UML diagrams are a standard visual notation to express object-oriented software designs. Several computer-aided tools are available to build UML diagrams. One type of UML figure is known as a **class diagram**.

An example of a class diagram is given in Figure 1.5, corresponding to our CreditCard class from Section 1.7. The diagram has three portions, with the first designating the name of the class, the second designating the recommended instance variables, and the third designating the recommended methods of the class. The type declarations of variables, parameters, and return values are specified in the appropriate place following a colon, and the visibility of each member is designated on its left, with the “+” symbol for **public** visibility, the “#” symbol for **protected** visibility, and the “-” symbol for **private** visibility.

class:	CreditCard	
fields:	- customer : String - limit : int - bank : String # balance : double - account : String	
methods:	+ getCustomer() : String + getAccount() : String + getBank() : String + getLimit() : int + charge(price : double) : boolean + getBalance() : double + makePayment(amount : double)	

Figure 1.5: A UML Class diagram for the CreditCard class from Section 1.7.

1.9.2 Pseudocode

As an intermediate step before the implementation of a design, programmers are often asked to describe algorithms in a way that is intended for human eyes only. Such descriptions are called *pseudocode*. Pseudocode is not a computer program, but is more structured than usual prose. It is a mixture of natural language and high-level programming constructs that describe the main ideas behind a generic implementation of a data structure or algorithm. Because pseudocode is designed for a human reader, not a computer, we can communicate high-level ideas without being burdened by low-level implementation details. At the same time, we should not gloss over important steps. Like many forms of human communication, finding the right balance is an important skill that is refined through practice.

There really is no precise definition of the pseudocode language. At the same time, to help achieve clarity, pseudocode mixes natural language with standard programming language constructs. The programming language constructs that we choose are those consistent with modern high-level languages such as C, C++, and Java. These constructs include the following:

- **Expressions:** We use standard mathematical symbols to express numeric and boolean expressions. To be consistent with Java, we use the equal sign “=” as the assignment operator in assignment statements, and the “==” relation to test equivalence in boolean expressions.
- **Method declarations:** **Algorithm** *name(param1, param2, ...)* declares new method “name” and its parameters.
- **Decision structures:** **if** condition **then** true-actions [**else** false-actions]. We use indentation to indicate what actions should be included in the true-actions and false-actions.
- **While-loops:** **while** condition **do** actions. We use indentation to indicate what actions should be included in the loop actions.
- **Repeat-loops:** **repeat** actions **until** condition. We use indentation to indicate what actions should be included in the loop actions.
- **For-loops:** **for** variable-increment-definition **do** actions. We use indentation to indicate what actions should be included among the loop actions.
- **Array indexing:** $A[i]$ represents the i^{th} cell in the array A . The cells of an n -celled array A are indexed from $A[0]$ to $A[n - 1]$ (consistent with Java).
- **Method calls:** `object.method(args)`; `object` is optional if it is understood.
- **Method returns:** **return** value. This operation returns the value specified to the method that called this one.
- **Comments:** { Comment goes here. }. We enclose comments in braces.

1.9.3 Coding

One of the key steps in implementing an object-oriented program is coding the descriptions of classes and their respective data and methods. In order to accelerate the development of this skill, we will discuss various *design patterns* for designing object-oriented programs (see Section 2.1.3) at various points throughout this text. These patterns provide templates for defining classes and the interactions between these classes.

Once we have settled on a design for the classes or our program and their responsibilities, and perhaps drafted pseudocode for their behaviors, we are ready to begin the actual coding on a computer. We type the Java source code for the classes of our program by using either an independent text editor (such as emacs, WordPad, or vi), or the editor embedded in an *integrated development environment* (IDE), such as Eclipse.

Once we have completed coding for a class (or package), we compile this file into working code by invoking a compiler. If we are not using an IDE, then we compile our program by calling a program, such as `javac`, on our file. If we are using an IDE, then we compile our program by clicking the appropriate compilation button. If we are fortunate, and our program has no syntax errors, then this compilation process will create files with a `.class` extension.

If our program contains syntax errors, then these will be identified, and we will have to go back into our editor to fix the offending lines of code. Once we have eliminated all syntax errors, and created the appropriate compiled code, we can run our program by either invoking a command, such as `java` (outside an IDE), or by clicking on the appropriate “run” button (within an IDE). When a Java program is run in this way, the runtime environment locates the directories containing the named class and any other classes that are referenced from this class according to a special operating system environment variable named `CLASSPATH`. This variable defines an order of directories in which to search, given as a list of directories, which are separated by colons in Unix/Linux or semicolons in DOS/Windows. An example `CLASSPATH` assignment in the DOS/Windows operating system could be the following:

```
SET CLASSPATH=.;C:\java;C:\Program Files\Java\
```

Whereas an example `CLASSPATH` assignment in the Unix/Linux operating system could be the following:

```
setenv CLASSPATH " ./usr/local/java/lib:/usr/netscape/classes"
```

In both cases, the dot (“.”) refers to the current directory in which the runtime environment is invoked.

1.9.4 Documentation and Style

Javadoc

In order to encourage good use of block comments and the automatic production of documentation, the Java programming environment comes with a documentation production program called *javadoc*. This program takes a collection of Java source files that have been commented using certain keywords, called *tags*, and it produces a series of HTML documents that describe the classes, methods, variables, and constants contained in these files. As an example, Figure 1.6 shows a portion of the documentation generated for our CreditCard class.

Each javadoc comment is a block comment that starts with “/**” and ends with “*/”, and each line between these two can begin with a single asterisk, “*”, which is ignored. The block comment is assumed to start with a descriptive sentence, which is followed by special lines that begin with javadoc tags. A block comment that comes just before a class definition, instance variable declaration, or method definition is processed by javadoc into a comment about that class, variable, or method. The primary javadoc tags that we use are the following:

- `@author text`: Identifies each author (one per line) for a class.
- `@throws exceptionName description`: Identifies an error condition that is signaled by this method (see Section 2.4).
- `@param parameterName description`: Identifies a parameter accepted by this method.
- `@return description`: Describes the return type and its range of values for a method.

There are other tags as well; the interested reader is referred to online documentation for javadoc for further information. For space reasons, we cannot always include javadoc-style comments in all the example programs included in this book, but we include such a sample in Code Fragment 1.8, and within the online code at the website that accompanies this book.

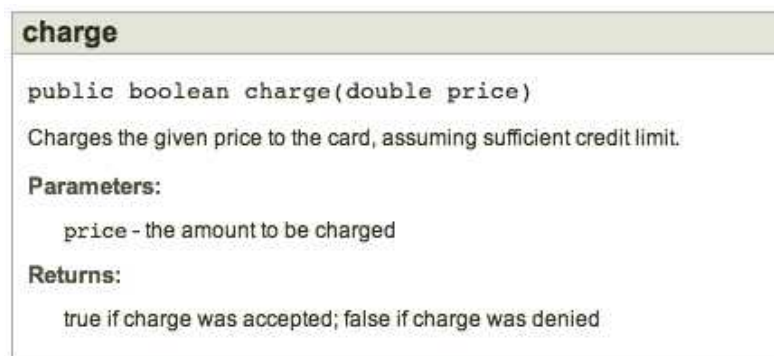


Figure 1.6: Documentation rendered by javadoc for the CreditCard.charge method.

```

1  /**
2   * A simple model for a consumer credit card.
3   *
4   * @author Michael T. Goodrich
5   * @author Roberto Tamassia
6   * @author Michael H. Goldwasser
7   */
8  public class CreditCard {
9      /**
10     * Constructs a new credit card instance.
11     * @param cust    the name of the customer (e.g., "John Bowman")
12     * @param bk      the name of the bank (e.g., "California Savings")
13     * @param acct    the account identifier (e.g., "5391 0375 9387 5309")
14     * @param lim     the credit limit (measured in dollars)
15     * @param initialBal the initial balance (measured in dollars)
16     */
17     public CreditCard(String cust, String bk, String acct, int lim, double initialBal) {
18         customer = cust;
19         bank = bk;
20         account = acct;
21         limit = lim;
22         balance = initialBal;
23     }
24
25     /**
26     * Charges the given price to the card, assuming sufficient credit limit.
27     * @param price    the amount to be charged
28     * @return true    if charge was accepted; false if charge was denied
29     */
30     public boolean charge(double price) {           // make a charge
31         if (price + balance > limit)                // if charge would surpass limit
32             return false;                          // refuse the charge
33         // at this point, the charge is successful
34         balance += price;                           // update the balance
35         return true;                                // announce the good news
36     }
37
38     /**
39     * Processes customer payment that reduces balance.
40     * @param amount   the amount of payment made
41     */
42     public void makePayment(double amount) {        // make a payment
43         balance -= amount;
44     }
45     // remainder of class omitted...

```

Code Fragment 1.8: A portion of the CreditCard class definition, originally from Code Fragment 1.5, with javadoc-style comments included.

Readability and Programming Conventions

Programs should be made easy to read and understand. Good programmers should therefore be mindful of their coding style, and develop a style that communicates the important aspects of a program's design for both humans and computers. Much has been written about good coding style, with some of the main principles being the following:

- Use meaningful names for identifiers. Try to choose names that can be read aloud, and choose names that reflect the action, responsibility, or data each identifier is naming. The tradition in most Java circles is to capitalize the first letter of each word in an identifier, except for the first word for a variable or method name. By this convention, “Date,” “Vector,” “DeviceManager” would identify classes, and “isFull(),” “insertItem(),” “studentName,” and “studentHeight” would respectively identify methods and variables.
- Use named constants or enum types instead of literals. Readability, robustness, and modifiability are enhanced if we include a series of definitions of named constant values in a class definition. These can then be used within this class and others to refer to special values for this class. The tradition in Java is to fully capitalize such constants, as shown below:

```
public class Student {  
    public static final int MIN_CREDITS = 12; // min credits per term  
    public static final int MAX_CREDITS = 24; // max credits per term  
    public enum Year {FRESHMAN, SOPHOMORE, JUNIOR, SENIOR};  
  
    // Instance variables, constructors, and method definitions go here...  
}
```

- Indent statement blocks. Typically programmers indent each statement block by 4 spaces; in this book we typically use 2 spaces, however, to avoid having our code overrun the book's margins.
- Organize each class in the following order:
 1. Constants
 2. Instance variables
 3. Constructors
 4. Methods

We note that some Java programmers prefer to put instance variable definitions last. We put them earlier so that we can read each class sequentially and understand the data each method is working with.

- Use comments that add meaning to a program and explain ambiguous or confusing constructs. In-line comments are good for quick explanations and do not need to be sentences. Block comments are good for explaining the purpose of a method and complex code sections.

1.9.5 Testing and Debugging

Testing is the process of experimentally checking the correctness of a program, while debugging is the process of tracking the execution of a program and discovering the errors in it. Testing and debugging are often the most time-consuming activity in the development of a program.

Testing

A careful testing plan is an essential part of writing a program. While verifying the correctness of a program over all possible inputs is usually infeasible, we should aim at executing the program on a representative subset of inputs. At the very minimum, we should make sure that every method of a program is tested at least once (method coverage). Even better, each code statement in the program should be executed at least once (statement coverage).

Programs often tend to fail on *special cases* of the input. Such cases need to be carefully identified and tested. For example, when testing a method that sorts (that is, puts in order) an array of integers, we should consider the following inputs:

- The array has zero length (no elements).
- The array has one element.
- All the elements of the array are the same.
- The array is already sorted.
- The array is reverse sorted.

In addition to special inputs to the program, we should also consider special conditions for the structures used by the program. For example, if we use an array to store data, we should make sure that boundary cases, such as inserting or removing at the beginning or end of the subarray holding data, are properly handled.

While it is essential to use handcrafted test suites, it is also advantageous to run the program on a large collection of randomly generated inputs. The `Random` class in the `java.util` package provides several means for generating pseudorandom numbers.

There is a hierarchy among the classes and methods of a program induced by the caller-callee relationship. Namely, a method *A* is above a method *B* in the hierarchy if *A* calls *B*. There are two main testing strategies, *top-down* testing and *bottom-up* testing, which differ in the order in which methods are tested.

Top-down testing proceeds from the top to the bottom of the program hierarchy. It is typically used in conjunction with *stubbing*, a bootstrapping technique that replaces a lower-level method with a *stub*, a replacement for the method that simulates the functionality of the original. For example, if method *A* calls method *B* to get the first line of a file, when testing *A* we can replace *B* with a stub that returns a fixed string.

Bottom-up testing proceeds from lower-level methods to higher-level methods. For example, bottom-level methods, which do not invoke other methods, are tested first, followed by methods that call only bottom-level methods, and so on. Similarly a class that does not depend upon any other classes can be tested before another class that depends on the former. This form of testing is usually described as **unit testing**, as the functionality of a specific component is tested in isolation of the larger software project. If used properly, this strategy better isolates the cause of errors to the component being tested, as lower-level components upon which it relies should have already been thoroughly tested.

Java provides several forms of support for automated testing. We have already discussed how a class's static main method can be repurposed to perform tests of the functionality of that class (as was done in Code 1.6 for the CreditCard class). Such a test can be executed by invoking the Java virtual machine directly on this secondary class, rather than on the primary class for the entire application. When Java is started on the primary class, any code within such secondary main methods will be ignored.

More robust support for automation of unit testing is provided by the JUnit framework, which is not part of the standard Java toolkit but freely available at www.junit.org. This framework allows the grouping of individual test cases into larger test suites, and provides support for executing those suites, and reporting or analyzing the results of those tests. As software is maintained, **regression testing** should be performed, whereby automation is used to re-execute all previous tests to ensure that changes to the software do not introduce new bugs in previously tested components.

Debugging

The simplest debugging technique consists of using **print statements** to track the values of variables during the execution of the program. A problem with this approach is that eventually the print statements need to be removed or commented out, so they are not executed when the software is finally released.

A better approach is to run the program within a **debugger**, which is a specialized environment for controlling and monitoring the execution of a program. The basic functionality provided by a debugger is the insertion of **breakpoints** within the code. When the program is executed within the debugger, it stops at each breakpoint. While the program is stopped, the current value of variables can be inspected. In addition to fixed breakpoints, advanced debuggers allow specification of **conditional breakpoints**, which are triggered only if a given expression is satisfied.

The standard Java toolkit includes a basic debugger named `jdb`, which has a command-line interface. Most IDEs for Java programming provide advanced debugging environments with graphical user interfaces.